1. Purpose, scope and deliverables

Real-Time Fraud Detection and Risk Scoring Platform

Portfolio project guide | Kavali Harshavardhan

This project demonstrates a real-time fraud-risk workflow for a portfolio. Incoming transactions receive a numeric risk score and an approve, verify or review decision. The decisions are simulated; the application does not execute payments, send OTPs or block accounts.

The working local path uses Python, Pandas, scikit-learn, FastAPI, SQLite and Streamlit. SQLite removes the need to configure a database server for the first demonstration. The optional Kafka bridge can connect an existing broker to the same API.

Included: 12,000 synthetic transactions, shared feature code, trained model, validation and holdout reports, API, dashboard, simulator, SQL, notebook, automated tests, Docker configuration, Excel workbook, diagrams and interview notes.

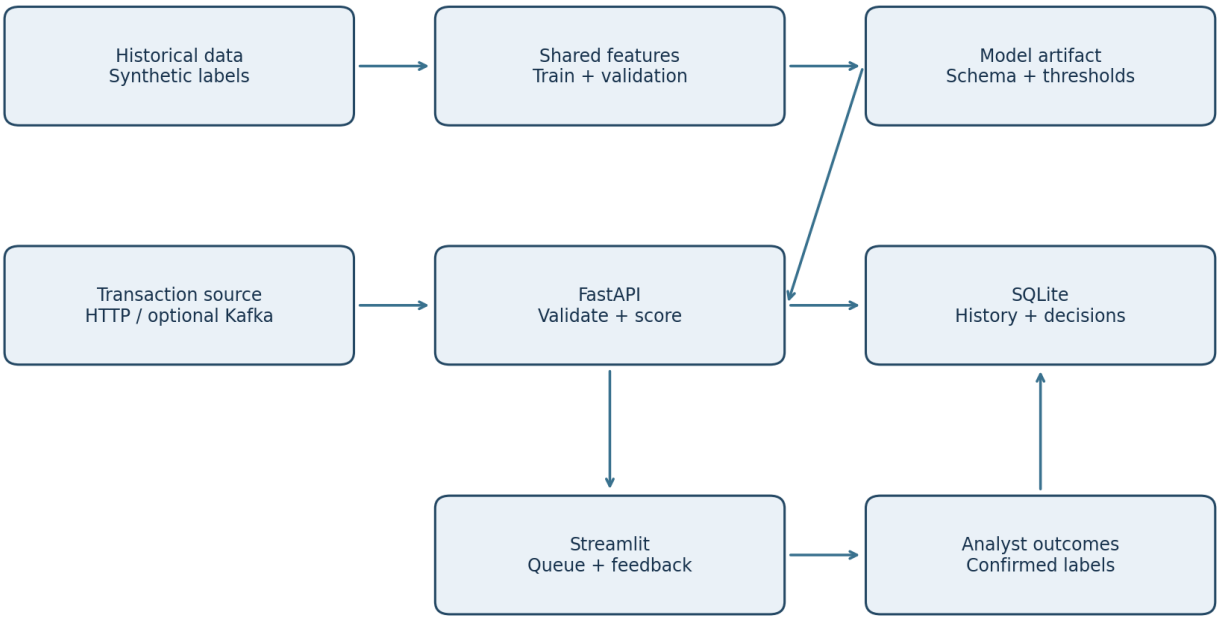
All events, customers and fraud labels are synthetic. Results explain this artificial dataset only. The project is a complete local portfolio demonstration; production deployment and a live Kafka broker remain separate operational work.

2. Architecture and complete workflow

Data flows through two paths. Offline training generates transactions, validates them, computes features in time order, trains candidates and evaluates a reserved holdout. Online scoring uses the same feature function, retrieves prior customer events from SQLite, loads the saved model and thresholds, then persists a decision.

The dashboard calls the API. It shows the latest 200 events and refreshes every three seconds. Its counts are explicitly scoped to these visible events. The analyst can record a confirmed outcome, which is stored separately from the original risk decision.

Workflow: define request and fraud outcome; generate data; validate; explore; calculate historical features; split by time; train; choose thresholds; evaluate; start API; replay transactions; review dashboard; collect outcomes; monitor shifts; retrain offline.



3. Data and feature engineering

Each transaction contains transaction_id, customer_id, timestamp, amount, device_id, country and home_country. Timestamps must include a timezone and are normalized to UTC. Amounts are INR and must be positive. is_fraud appears only in historical labeled data and is rejected as an extra API input.

The generator uses seed 42 and 30-second event spacing. Customers, amounts, device changes and foreign-country events are sampled. Fraud labels are Bernoulli samples from a declared artificial risk function, not real chargeback data. This limits generalization and deliberately avoids a perfectly separable label.

Features: log(1+amount); amount divided by the preceding 24-hour mean; number of customer events in the preceding hour; unseen device indicator relative to earlier customer devices; UTC hour before 06:00; country differing from home country.

Only strictly earlier events are eligible for history. The current event is appended after feature calculation. For no recent history, the amount ratio is one. A first-ever device is not called new until earlier customer history exists. UTC night is a teaching simplification, not the customer local timezone.

Input validation checks positive amounts, finite values, identifier length, country-code shape and timezone presence. This is format validation, not full ISO-country membership or account-ownership verification.

4. Modeling and evaluation

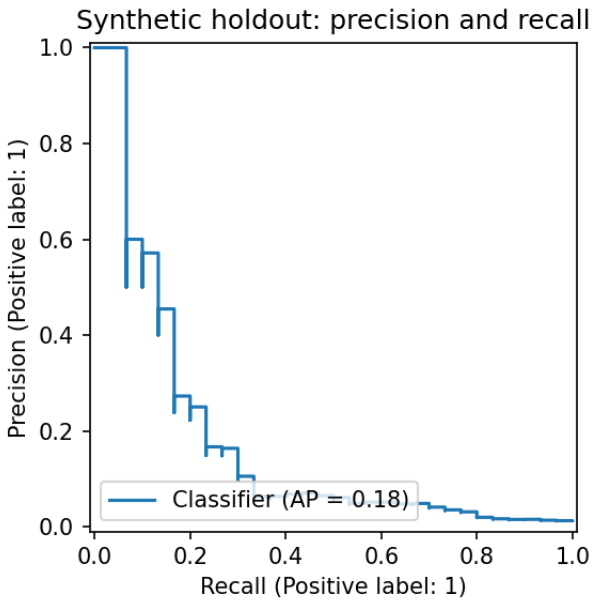
The earliest 7,200 rows train the models. The next 2,400 choose the model and thresholds. The final 2,400 are the holdout. Earlier event history can cross split boundaries because it would have existed at scoring time; fraud labels never become historical features.

Candidates are Logistic Regression with training-only standardization and Random Forest with bounded depth and leaf size. The highest validation average precision wins. No oversampling is applied. The chosen model is Random Forest.

The verify threshold maximizes validation F2 over a fixed grid. F2 gives recall more emphasis than precision. The review threshold uses the larger of the validation 95th percentile and verify threshold plus 0.001. It targets an approximate high-priority queue size, not a guaranteed future service level.

On the synthetic holdout, average precision is 0.1807, precision is 5.59%, recall is 53.33%, false-positive rate is 11.39%, and alert rate is 11.92%. There are 16 true positives, 270 false positives, 14 false negatives and 2,100 true negatives.

The result demonstrates a trade-off: 286 events are flagged to capture 16 of 30 fraud events. The false-alert burden is substantial. Captured fraud amount is 86.49% of labeled fraud amount; this is not money saved. Human intervention cost and real loss avoidance are not measured.



5. Risk decisions, persistence and API

Score equals 100 times the model output. At this trained version, verify begins at model output 0.025 and review at about 0.05734. Scores are rounded only for display; decisions use the unrounded output. A score is not a calibrated fraud probability and a high score does not prove fraud.

GET /health reports service and model status. POST /score accepts a transaction. GET /transactions returns the latest 200 stored events. POST /feedback records a binary confirmed outcome for an existing transaction. Interactive schemas are available at /docs when the API is running.

An atomic SQLite transaction covers history read and decision insert. transaction_id is a primary key. An identical retry returns the original result. A reused ID with a changed payload returns HTTP 409. Strictly non-increasing event times for one customer also return 409.

Reason flags indicate observed conditions such as a new device or elevated amount. They are not SHAP values or causal explanations. Feedback accepts 0 or 1 and uses a foreign key. Unreviewed events are not assumed legitimate.

Only one small local writer workload is intended. SQLite locks serialize writes and full customer-history scans grow with usage. A larger system needs bounded feature storage, consistent device state, managed schema migrations and load-tested concurrency.

6. VS Code setup and demonstration

Install Python 3.12 and extract the ZIP. Open fraud-platform in VS Code. In its terminal run python -m venv .venv. Activate with .venv\Scripts\Activate.ps1 on Windows PowerShell or source .venv/bin/activate on macOS/Linux. If PowerShell blocks activation, use .venv\Scripts\python.exe directly rather than changing machine policy.

Run python -m pip install -r requirements.txt. Run python -m src.train to regenerate a model compatible with your installed libraries. Run python -m pytest -q. Then start python -m uvicorn src.api:app --host 127.0.0.1 --port 8000.

In a second activated terminal run python -m streamlit run dashboard.py and open http://localhost:8501. In a third activated terminal run python -m src.simulate --count 300 --delay 0.1. Observe scores and the review queue, then enter a transaction manually.

Use the API documentation at http://localhost:8000/docs to replay an ID and inspect validation errors. Record an outcome through the dashboard. SQLite state lives in data/live.db by default.

Replay begins at historical time. If a customer already has a current-time manual event, older replay events for that customer are rejected. Use a new FRAUD_DB path for a clean demonstration. Retraining changes the saved artifact; restart the API to load it.

7. Tests, monitoring and deployment boundaries

Automated tests cover scoring, idempotent retries, conflicting payloads, out-of-order transactions, invalid amounts, naive timestamps, malformed country codes, unknown fields, feedback existence, binary outcomes and future-event exclusion. See reports/test_results.txt for the actual run.

The live check launches the API, replays 300 transactions and executes the Streamlit application using its test harness. reports/live_check.json records results and local scoring timings. These timings exclude the final database insert/commit and network round trip; they are not production latency claims.

src/monitor.py compares training and holdout feature means and reports absolute standardized shifts. It is a descriptive batch check. It does not implement automated alerts, PSI thresholds or drift-triggered retraining. Monitor delayed-label precision, recall, queue capacity, errors, latency and feature distributions in a future operational system.

Dockerfile and compose.yaml provide local deployment configuration and persistent storage. Docker execution is not validated here. Kafka adapter code is supplied; the external broker path is not integration-tested here. Public hosting is not included.

Before production use, add authentication and authorization, encrypted transport, rate limits, secrets handling, real-time feature consistency, monitoring, backups, calibration, cost-aware policies and validation on approved real data. This demo binds to local interfaces and contains no real personal data.

8. Project structure, GitHub and interview use

src/ owns generation, features, training, API, scoring, simulation and optional transport. sql/ contains the schema and queries. tests/ validates behavior. models/ contains the trusted demonstration artifact. data/ contains synthetic source and derived features. notebooks/ contains EDA. reports/ contains metrics and Excel. docs/ contains this guide, model card and diagrams.

Open reports/Fraud_Analytics.xlsx for a formula-linked holdout summary, predictions and dictionary. Counts and ratios are derived from included predictions. The workbook is an analysis snapshot, not connected to the live API. Run the EDA notebook to explore data and consult source scripts for exact implementation.

The repository includes a GitHub Actions workflow to install dependencies, train and run tests. A pushed workflow is configuration, not evidence of a completed remote CI run. Review Actions after GitHub upload.

In interviews, explain the problem, architecture, chronological split, duplicate handling, cold start, threshold trade-off and synthetic-data limitations. Do not claim prevented losses, real banking deployment or production throughput. Use docs/INTERVIEW.md for a five-minute demo and resume bullets.

Reproduction: python -m src.generate; python -m src.train; python -m src.monitor; python -m pytest -q. README.md includes startup instructions. The PDF and Excel are delivered snapshots; rerunning training refreshes machine-readable reports and plots, while authored documentation must be updated to match new results.